We'll make a couple of points about the include statement before we go on.

First, the include statement has nothing to do with files. The Ruby include statement simply makes a reference to a module. If that module is in a separate file, you must use require or require_relative to drag that file in before using include. The require call is at the the file level, and loads the module into the Ruby application as whole, the include call is at the class level, and adds the module's behavior to the class in which it is included.

Second, a Ruby include does not copy and paste the module's instance methods into the class. Instead, it makes a reference from the class to the included module. If multiple classes include that module, they'll all point to the same thing. If you change the definition of a method within a module, even while your program is running, all classes that include that module will exhibit the new behavior. Of course, we're speaking only of methods here. Instance variables are always different per object.

Mixins give you a wonderfully controlled way of adding functionality to classes. However, their true power comes out when the code in the mixin can make assumptions about code in the class that uses it and then can interact with that code.

Ruby uses mixin behavior in the standard library extensively. Many of the behaviors we have seen that are available to all objects are actually defined in a module called Kernel which is included into Object and therefore of all objects. Methods like puts, p, lambda, proc and many more are added to objects using mixin behavior.

The standard Ruby module Comparable is another great example of a mixin, but one that makes an assumption about the classes that use it. Including Comparable as a mixin adds the comparison operators (<, <=, ==, >=, and >), as well as the method between?, to a class. For these methods to work, Comparable assumes that any class that uses it defines the method <=>, also known as the "spaceship operator". The spaceship operator compares two values, returning -1, 0, or +1 depending on whether the first is less than, equal to, or greater than the second, respectively. As a class writer, you can define one method, <=>; include Comparable; and get six comparison functions for free.

Let's take a Person class. We'll make people comparable based on their names:

```
class Person
  include Comparable
  attr_reader :name

  def initialize(name)
    @name = name
  end

  def to_s
    "#{@name}"
  end

  def <=>(other)
    self.name <=> other.name
  end
end

p1 = Person.new("Matz")
p2 = Person.new("Guido")
p3 = Person.new("Larry")

if p1 > p2
```